Resume Parser & Job-Role Matching AI Engine

Team:

D. Vinay – 2300080280
G. Moulika – 2300080066
J. Navya - 2300080019

Contents

1. Abstract
2. Objectives
3. Problem Statement
4. Literature Review
5. Methodology
6. Evaluation & Results
7. Implementation & Output
8. Limitations / Challenges
9. Conclusion

Abstract

This project presents a Resume Parser and Job-Role Matching System to improve recruitment efficiency. The system extracts structured information such as Name, Email, Phone, and Skills using rule-based techniques and a predefined skill database. It then applies Sentence-BERT embeddings and cosine similarity to compute semantic compatibility between resumes and job descriptions. The system provides explainable match scores and classification into Strong, Moderate, or Weak fit. Future enhancements include expanding the skill database and incorporating advanced NLP models.

Objectives

- Automate resume parsing across varied formats.
- Develop a semantic job-role matching engine beyond keyword-based methods.
- Reduce recruiter effort and time-to-hire with scalable, auditable systems.
- Ensure explainability and trust in AI-driven hiring decisions.

Problem Statement

- Hiring at scale is time-consuming and error-prone.
- Recruiters manually screen hundreds of resumes.
- Resumes are unstructured: varied formats, synonyms, and implicit skills.
- Job-role fit requires semantic understanding beyond keyword matching.

Goal: Build an automated system that parses resumes accurately and ranks candidates by semantic fit to job roles.

Problem Justification

- Large companies receive hundreds of resumes per job posting
- Recruiters spend 6–8 seconds per resume during initial screening
- Manual screening increases time-to-hire and inconsistency
- Traditional ATS systems rely on keyword matching and lack semantic understanding

Sources: LinkedIn Hiring Reports, SHRM HR Studies, ATS Research Literature

Literature Review

- **Traditional ATS** → keyword-based, poor semantic understanding.
- **ML / Early NLP** → TF-IDF, n-grams; limited accuracy on varied formats.
- **Modern NLP & embeddings** → improved semantic matching.
- **Ontologies & synonym mapping** → better skill extraction.
- **Limitations** → Data scarcity, high compute, bias, low explainability.
- **Research Gap** → Lack of contextual semantic matching combined with transparent scoring.

Proposed System – Improvements Over Existing Methods

Feature	Traditional ATS	Basic NLP (TF-IDF)	Our System
Keyword Matching	✓	✓	✓
Semantic Understanding	✕	Limited	✓ (SBERT)
Context Awareness	✕	✕	✓
Skill Overlap Ratio	Limited	Limited	✓
Explainable Match Score	✕	✕	✓
Lightweight Deployment	Limited	Limited	✓ (Streamlit)

Methodology

Resume Parsing:
- PDF text extraction using pdfplumber
- Name, Email, Phone extracted using regex
- Skills detected using predefined skill database

Job Matching:
- Sentence-BERT embeddings for resume & job description
- Cosine similarity to compute match score (%)
- Skill Match Ratio for direct overlap analysis

Workflow:
Upload → Text Extraction → Parsing → Embedding → Similarity → Match Result

Explainability:
- Displays extracted details
- Provides Strong / Moderate / Weak classification

Evaluation & Results

Parsing Performance:
- Extracts Name, Email, Phone using regex
- Identifies technical skills from predefined database

Matching Performance:
- Uses Sentence-BERT embeddings
- Cosine similarity computes match score (%)
- Categorizes into Strong / Moderate / Weak match

Results & Impact:
- Example: ML Resume → ML JD = 82%
  ML Resume → Web JD = 47%
- Skill Match Ratio highlights direct skill overlap
- Reduces manual resume screening effort

Implementation – Semantic Matching Engine

```
model = SentenceTransformer("all-MiniLM-L6-v2")

resume_embedding = model.encode([resume_text])
job_embedding = model.encode([job_description])

similarity = cosine_similarity(resume_embedding,
job_embedding)[0][0]
score = round(similarity * 100, 2)
```

Implementation – Skill Extraction

```
SKILL_DATABASE = [
    "Python", "Java", "C", "Spring Boot", "React",
    "MySQL", "PostgreSQL", "Machine Learning",
    "Deep Learning", "NLP", "Cloud Computing"
]

def extract_skills(text):
    return [skill for skill in SKILL_DATABASE
            if skill.lower() in text.lower()]
```

Implementation – Resume Parsing

```
def extract_email(text):
    match = re.search(r"...", text)
    return match.group(0) if match else "Not Found"


def extract_phone(text):
    match = re.search(r"\b\d{10}\b", text)
    return match.group(0) if match else "Not Found"
```

Output

📌 Extracted Resume Information

Name: Rahul Sharma

Email: rahul.sharma.dev@gmail.com

Phone: 9123456789

Skills Found: Python, Java, C, Spring Boot, React, MySQL, PostgreSQL, Machine Learning, NLP, Cloud Computing, JWT, REST API

🎯 Matched Skills with Job Description

MySQL, PostgreSQL, C, Java, REST API, Python

Skill Match Ratio: 6 / 6

📊 Semantic Match Score

60.44%

🟡 Moderate Match

Limitations / Challenges

Expected limitations:

- Skill detection limited to predefined skill database.
- Regex-based parsing may fail for complex resume formats.
- Does not include large-scale dataset evaluation.
- System currently supports single resume-job comparison.

Conclusion & Future Work

- Successfully implemented resume parsing using rule-based extraction.
- Developed semantic job-role matching using SBERT embeddings.
- Provides interpretable and transparent semantic match scoring.
- Future work: expand skill ontology, integrate larger datasets, and enhance extraction accuracy.

THANK YOU